ShopSense — Exam & Viva Master Preparation Guide

Essential Theory, Architecture, Algorithms, Database Design & Viva Q&A; | Candidate: Shaurya

Subject: E-Commerce System & Database Engineering	**Target Tech:** PostgreSQL, Supabase, Python, Node.js
Core Topics: ERD, Normalization, Search (BM25+RRF), Recommenders (CF+MF)	**Database:** Live Cloud (nzccgolsbffelzdxtkom.supabase.co)

Module 1: Database Architecture, ER Diagrams & Normalization

1.1 What is an Entity-Relationship (ER) Diagram?

An ER Diagram visually models the data requirements of a system using three core constructs:

• **Entities**: Real-world objects represented as tables (e.g., USERS, PRODUCTS, ORDERS).

• **Attributes**: Properties describing an entity (e.g., price, email, status).

• **Relationships & Cardinality**: Associations between entities (e.g., 1 User places N Orders ➔ **1:N**; Orders and Products connect via **M:N** junction tables).

1.2 Weak Entities & Composite Primary Keys

• **Weak Entity:** An entity that cannot be uniquely identified by its own attributes alone and depends on a foreign key from a parent entity.

• In ShopSense, CART_ITEMS, WISHLIST_ITEMS, and ORDER_ITEMS are associative weak entities.

• **Composite Primary Key:** A primary key consisting of two or more columns. In cart_items, the composite PK (user_id, product_id) ensures a customer cannot have duplicate entries for the same product; instead, quantity is incremented.

1.3 Database Normalization (Applied in ShopSense)

• **1NF (First Normal Form):** All column values are atomic (no arrays/repeating groups in relational columns). Primary key is defined.

• **2NF (Second Normal Form):** Must be in 1NF and have no partial dependencies (every non-key attribute must depend on the whole composite key). For example, in order_items, attributes like quantity depend on both (order_id, product_id).

• **3NF (Third Normal Form):** Must be in 2NF and have no transitive dependencies (non-key attributes do not depend on other non-key attributes). Product metadata belongs in products, not duplicated in cart_items.

1.4 Why Supabase (Cloud PostgreSQL)?

• **PostgreSQL Power:** Enterprise relational database with full ACID compliance, foreign key constraints, indexes, and JSONB support.

• **Row Level Security (RLS):** Enforces security policies at the database engine level (e.g., auth.uid() = user_id). Even if an API bug occurs, users cannot access other users' carts or orders.

• **Cloud Persistence:** Solves the ephemeral container disk problem (e.g., on Render or Heroku) by persisting data safely in the cloud.

Module 2: Search Engine & Information Retrieval (IR)

ShopSense implements a custom **hybrid search engine** built from algorithmic first principles without external search servers:

2.1 Inverted Index & Tokenization Pipeline

• **Tokenization:** Breaking text into individual words, converting to lowercase, and stripping punctuation.
• **Stemming:** Reducing inflected words to their root form (e.g., 'running', 'runs', 'ran' ➔ 'run') using algorithmic rule sets.
• **Inverted Index:** A mapping from terms to the document IDs containing them: Term ➔ [Doc1, Doc5, Doc12]. Allows looking up matching documents in **O(1)** average time rather than scanning all products **O(N)**.

2.2 BM25 (Best Matching 25) Keyword Ranking

• State-of-the-art probabilistic relevance ranking algorithm used by industrial search engines.
• Evaluates term saturation using non-linear TF: multiple occurrences of a word provide diminishing returns.
• Document Length Normalization: Penalizes very long titles/descriptions to prevent spamming keywords.
• Formula components: IDF(q_i) * (TF * (k1 + 1)) / (TF + k1 * (1 - b + b * (|D| / avgdl))).

2.3 Vector Space Model & Cosine Similarity

• Represents product documents and user queries as high-dimensional TF-IDF vectors.
• Calculates angle cosine: $\cos(\theta) = (A \cdot B) / (||A|| \, ||B||)$. Values close to 1 indicate strong topical similarity.

2.4 Reciprocal Rank Fusion (RRF)

• Combines disparate ranked lists (BM25 keyword search + Vector semantic search) without needing score calibration:
• $RRF_Score(d) = \Sigma [1 / (k + rank_i(d))]$ (where $k = 60$ is a smoothing constant).
• Products ranking high in both BM25 and vector spaces are boosted to the top of the search results.

2.5 Trie for Autocomplete & Levenshtein for Typo Tolerance

• **Trie (Prefix Tree):** Fast autocomplete suggestions in O(M) time (where M is query length), independent of catalog size.
• **Levenshtein Distance:** Dynamic programming edit distance calculating the minimum insertions, deletions, or substitutions to match a word. Corrects typos (e.g., 'shose' ➔ 'shoes').

Module 3: Recommendation Algorithms & Machine Learning

3.1 Cold-Start Problem

• **Problem:** New users have zero interaction history, making personalized collaborative filtering impossible.
• **ShopSense Solution:** Hybrid fallback. Unauthenticated or new users receive popularity-decayed trending items, curated category shelves, and demographic picks until their first interaction is logged.

3.2 Item-Item Collaborative Filtering (CF)

• Finds item relationships based on user behavior: 'Customers who viewed/bought item X also viewed item Y'.
• Computes similarity using co-occurrence matrices weighted by interaction types (purchases weighted higher than views).

3.3 Matrix Factorization (Latent Factor Model via SGD)

• Decomposes the sparse User-Item interaction matrix R (users × items) into two low-rank dense matrices: P (users × k) and Q (items × k).
• $Predicted_Rating(u, i) = P_u \cdot Q_i^T$.
• Trained using **Stochastic Gradient Descent (SGD)** to minimize squared reconstruction error with L2 regularization.

Module 4: Top 20 Viva / Oral Exam Questions & Model Answers

These questions represent the most frequently asked questions during technical viva evaluations. Study these high-scoring answers:

Q1: What is the main objective and architecture of ShopSense?

ShopSense is a full-stack product discovery e-commerce platform featuring hybrid search (BM25 + TF-IDF Vector Space + RRF), hybrid recommendations (CF + Matrix Factorization + Trending), and a cloud PostgreSQL database hosted on Supabase.

Q2: Explain the ER Diagram design for this project.

The ER diagram consists of 10 tables: USERS, PRODUCTS, and ORDERS as primary entities; CART_ITEMS, WISHLIST_ITEMS, and ORDER_ITEMS as associative junction tables with composite primary keys; and INTERACTIONS, EVENTS, SEARCH_HISTORY, and SEARCH_LOGS for telemetry and ML recommendations.

Q3: Why did you use composite primary keys in CART_ITEMS and ORDER_ITEMS?

A composite key (user_id, product_id) in cart_items prevents duplicate rows for the same product per user. It enforces uniqueness and integrity at the database schema level while simplifying updates to quantity.

Q4: What is the difference between SQLite and Supabase PostgreSQL?

SQLite is an embedded, single-node file database suitable for local development. Supabase is a cloud-hosted, enterprise PostgreSQL database with multi-user concurrency, network access, ACID transactions, and Row Level Security (RLS) suitable for production.

Q5: How is Row Level Security (RLS) implemented?

RLS policies are defined on cart_items, wishlist_items, and orders using `auth.uid() = user_id`. This guarantees that queries only return data belonging to the authenticated user, enforcing authorization directly in the database engine.

Q6: How does the BM25 search algorithm work?

BM25 is a probabilistic retrieval model that scores documents based on query term frequency with diminishing returns (term saturation) and normalizes for document length to prevent long product descriptions from artificially dominating search results.

Q7: What is Reciprocal Rank Fusion (RRF) and why is it used?

RRF is an ensemble ranking technique that merges ordered results from keyword search (BM25) and semantic vector search using the formula score = Σ [1 / (k + rank)]. It eliminates the need to normalize raw scores from different algorithms.

Q8: What data structure is used for autocomplete and why?

A Trie (prefix tree) is used. It allows prefix matching in O(M) time, where M is the prefix length, making suggestions virtually instantaneous regardless of how many thousands of terms exist in the catalog.

Q9: How does the system handle typos in user queries?

It computes Levenshtein Distance (minimum character insertions, deletions, and substitutions) using dynamic programming. If a query term has no exact matches, it suggests or substitutes the closest catalog terms within edit distance 1 or 2.

Q10: What is Matrix Factorization in recommendation systems?

Matrix Factorization decomposes a high-dimensional sparse user-item interaction matrix into two low-rank dense matrices representing latent user preferences and latent item features. Their dot product predicts user affinity for unseen items.

Q11: How do you address the Cold-Start problem?

For new users without interaction history, the system serves popularity-ranked trending products with time-decay scores, featured brand shelves, and curated lists until their first views or clicks are recorded.

Q12: How are passwords stored securely?

Passwords are never stored in plaintext. They are hashed using scrypt with a cryptographic salt (16 bytes) and 64-byte derived key, making them resistant to rainbow table and brute-force GPU attacks.

Q13: How does JWT authentication work?

A JSON Web Token consists of Header, Payload, and Signature encoded in Base64URL. The server verifies identity statelessly by validating the HMAC-SHA256 signature using a secret key without querying the database for session state.

Q14: Explain the role of the Python `database/` module.

The `database/` folder provides an object-oriented Python layer featuring dataclass models (models.py), repository classes for CRUD operations (repository.py), automated cloud migration (seed.py), and diagnostic verification (test_connection.py).

Q15: What happens when a user checks out their cart?

An atomic database transaction creates a new row in `orders`, moves all lines from `cart_items` into `order_items`, logs a 'purchase' interaction for the recommender, and clears the user's cart.

Q16: What HTTP methods and status codes are used?

Standard REST conventions: GET (200), POST (201 Created), PUT/PATCH (200), DELETE (200), 400 Bad Request, 401 Unauthorized, 404 Not Found, and 409 Conflict (e.g. duplicate email registration).

Q17: How is CI/CD configured for deployment?

Deployment is managed via Render Blueprints (`render.yaml`). Every `git push origin main` triggers an automated build webhook that installs dependencies, executes migrations, and starts the service with zero downtime.

Q18: What is the purpose of the `interactions` vs `events` tables?

`interactions` logs domain-specific signals (views, cart adds, purchases) strictly used by the collaborative filtering and matrix factorization engines. `events` records general client-side telemetry and audit logs.

Q19: How are database constraints enforced?

Foreign keys with `ON DELETE CASCADE` ensure child records (e.g. order items when an order is deleted) are cleaned up. Column check constraints (`quantity > 0`) prevent invalid transaction data.

Q20: What are your key contributions to this repository?

Designed and implemented the complete 10-table Supabase PostgreSQL cloud migration, built the modular Python database package, synchronized cloud persistence with the Node.js API, configured Render CI/CD, and authored 8 verified commits.